python

Python Data Structures

Dr. Arvind Keprate

Python Data Structures

- Lists
- Tuples
- Dictionaries

Python Data Structures

LEARNING OUTCOMES

Understand tuples and lists by describing and manipulating tuple combinations and list data structures.

Demonstrate understanding of dictionaries by writing structures with correct keys and values.

Understand the differences between sets, tuples, and lists by creating sets.

Lists

- In addition to number, Boolean, and string values, Python supports lists
- List is an ordered sequence which is mutable.

N = [1,2,3,4,5]

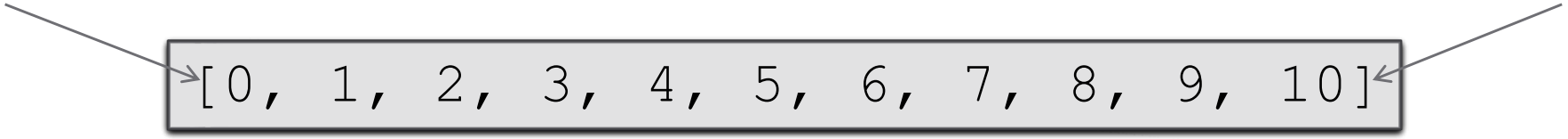
Vowels = [“a”, “e”, “i”, “o”, “u”]

- List can contain strings, numbers, another lists and tuples, e.g.

[“Norway”, 5.6, 1, [1,3], (“A”, 2)]

Lists

- A comma-separated sequence of items enclosed within square brackets



- The items can be numbers, strings, and even other lists

```
>>> pets = ['ant', 'bat', 'cod', 'dog', 'elk']
>>> lst = [0, 1, 'two', 'three', [4, 'five']]
>>> nums = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
>>>
```

Lists Indexing and Slicing

```
L =["Michael Jackson", 10.1,1982]
```

0	“Michael Jackson”
1	10.1
2	1982

L[0]: “Michael Jackson”

L[1]: 10.1

L[2]: 1982

```
L =["Michael Jackson", 10.1,1982]
```

-3	0	“Michael Jackson”
-2	1	10.1
-1	2	1982

L[-3]: “Michael Jackson”

L[-2]: 10.1

L[-1]: 1982

```
L =["Michael Jackson", 10.1,1982, “MJ”, 1]
```

L[3:5]:[“MJ”, 1]

Lists are mutable, strings are not

Lists can be modified; they are said to be mutable

```
pets = ['ant', 'bat', 'cow', 'dog', 'elk']
```

Strings can't be modified; they are said to be immutable

```
pet = 'cod'
```

```
>>> pets = ['ant', 'bat', 'cod', 'dog', 'elk']
>>> pets[2] = 'cow'
>>> pets
['ant', 'bat', 'cow', 'dog', 'elk']
>>> pet = 'cod'
>>> pet[2] = 'w'
Traceback (most recent call last):
  File "<pyshell#155>", line 1, in <module>
    pet[2] = 'w'
TypeError: 'str' object does not support item assignment
>>>
```

List operators and functions

Like strings, lists can be manipulated with operators and functions

Usage	Explanation
`x in lst`	`x` is an item of `lst`
`x not in lst`	`x` is not an item of `lst`
`lst + lstB`	Concatenation of `lst` and `lstB`
`lst*n, n*lst`	Concatenation of `n` copies of `lst`
`lst[i]`	Item at index `i` of `lst`
`len(lst)`	Number of items in `lst`
`min(lst)`	Minimum item in `lst`
`max(lst)`	Maximum item in `lst`
`sum(lst)`	Sum of items in `lst`

```
>>> lst = [1, 2, 3]
>>> lstB = [0, 4]
>>> 4 in lst
False
>>> 4 not in lst
True
>>> lst + lstB
[1, 2, 3, 0, 4]
>>> 2*lst
[1, 2, 3, 1, 2, 3]
>>> lst[0]
1
>>> lst[1]
2
>>> lst[-1]
3
>>> len(lst)
3
>>> min(lst)
1
>>> max(lst)
3
>>> sum(lst)
6
>>> help(list
...
```

Lists Concatenation

```
L=["Michael Jackson", 10.1,1982]
```

1st Way:

```
L1 = L+["pop",10]
```

```
L1=["Michael Jackson", 10.1,1982, "pop",10]
```

2nd Way:

```
L.extend(["pop",10])
```

```
L=["Michael Jackson", 10.1,1982, "pop",10]
```

3rd Way:

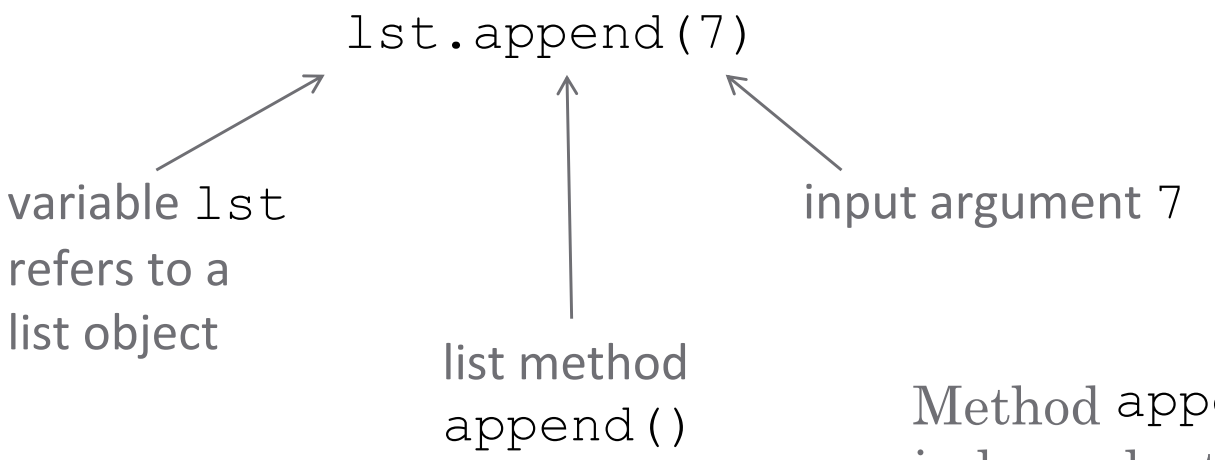
```
L.append (["pop",10])
```

```
L=["Michael Jackson", 10.1,1982, ["pop",10]]
```

Lists methods

`len()` and `sum()` are examples of functions that can be called with a list input argument; they can also be called on other type of input argument(s)

There are also functions that are called on a list; such functions are called list methods




```
>>> lst = [1, 2, 3]
>>> len(lst)
3
>>> sum(lst)
6
>>> lst.append(7)
>>> lst
[1, 2, 3, 7]
>>>
```

Method `append()` can't be called independently; it must be called on some list object

Lists methods

Usage	Explanation
`lst.append(item)`	adds `item` to the end of `lst`
`lst.count(item)`	returns the number of times `item` occurs in `lst`
`lst.index(item)`	Returns index of (first occurrence of) `item` in `lst`
`lst.pop()`	Removes and returns the last item in `lst`
`lst.remove(item)`	Removes (the first occurrence of) `item` from `lst`
`lst.reverse(item)`	Reverses the order of items in `lst`
`lst.sort(item)`	Sorts the items of `lst` in increasing order

Methods `append()`, `remove()`, `reverse()`, and `sort()` do not return any value; they, along with method `pop()`, modify list `lst`

```
>>> lst = [1, 2, 3]
>>> lst.append(7)
>>> lst.append(3)
>>> lst
[1, 2, 3, 7, 3]
>>> lst.count(3)
2
>>> lst.remove(2)
>>> lst
[1, 3, 7, 3]
>>> lst.reverse()
>>> lst
[3, 7, 3, 1]
>>> lst.index(3)
0
>>> lst.sort()
>>> lst
[1, 3, 3, 7]
>>> lst.remove(3)
>>> lst
[1, 3, 7]
>>> lst.pop()
7
>>> lst
[1, 3]
```

Trivia Time

Consider the following list **B=[1,2,[3,'a'],[4,'b']]**. What is the result of the following: **B[3][1]**

- ○ "c"
- ✗ "b"
- ○ [4,"b"]

What is the result of the following operation?
[1,2,3]+[1,1,1]

- ○ TypeError
- ✗ [1, 2, 3, 1, 1, 1]
- ○ [2,3,4]

Trivia Time

What is the length of the list **A = [1]** after the following operation: **A.append([2,3,4,5])**

- ✗ 2
- 5

What is the result of the following: **"Hello Mike".split()**

- ["H"]
- ["HelloMike"]
- ✗ ["Hello","Mike"]

Practice Session: Jupyter Notebook

Lecture_2_Lists_MorePractice

BREAK: 15Minz

Task 1: 10 minutes

List **`lst`** is a list of prices for a pair of boots at different online retailers:

`lst = [159.99, 160.00, 205.95, 128.83, 175.49]`

a) You found another retailer selling the boots for $160.00; add this price to list `lst`

b) Compute the number of retailers selling the boots for $160.00

c) Find the minimum price in `lst`

d) Using c), find the index of the minimum price in list `lst`

e) Using c) remove the minimum price from list `lst`

f) Sort list `lst` in increasing order

```
>>> lst = [159.99, 160.00, 205.95,
128.83, 175.49]

>>> lst.append(160.00)

>>> lst.count(160.00)
2

>>> min(lst)
128.83

>>> lst.index(128.83)
3

>>> lst.remove(128.83)

>>> lst
[159.99, 160.0, 205.95, 175.49, 160.0]

>>> lst.sort()

>>> lst
[159.99, 160.0, 160.0, 175.49, 205.95]
>>>
```

Tuple

The class `tuple` is the same as class list … except that it is immutable

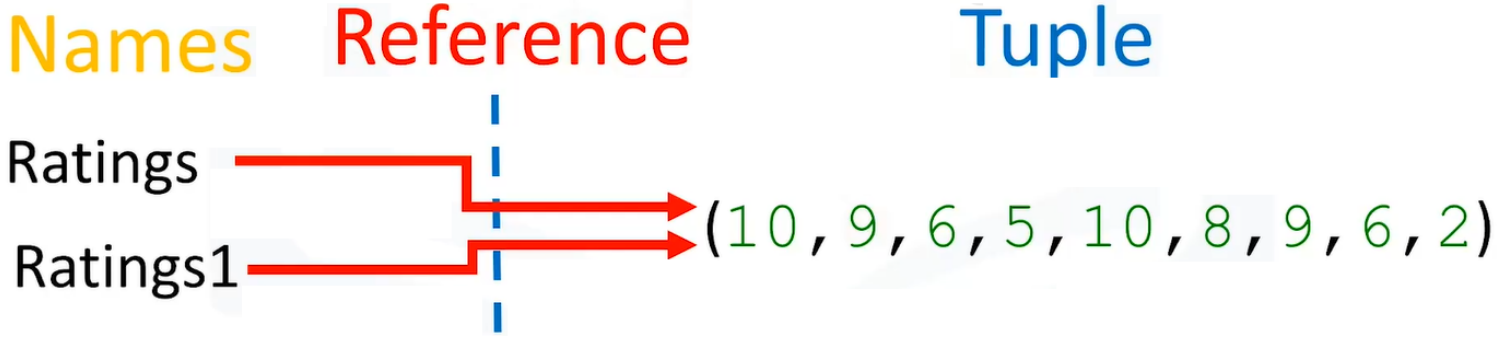
```
>>> lst = ['one', 'two', 3]
>>> lst[2]
3
>>> lst[2] = 'three'
>>> lst
['one', 'two', 'three']
>>> tpl = ('one', 'two', 3)
>>> tpl
('one', 'two', 3)
>>> tpl[2]
3
>>> tpl[2] = 'three'
Traceback (most recent call last):
  File "<pyshell#131>", line 1, in <module>
    tpl[2] = 'three'
TypeError: 'tuple' object does not support item assignment
>>>
```

Why do we need it? Sometimes, we need to have an “immutable list”.

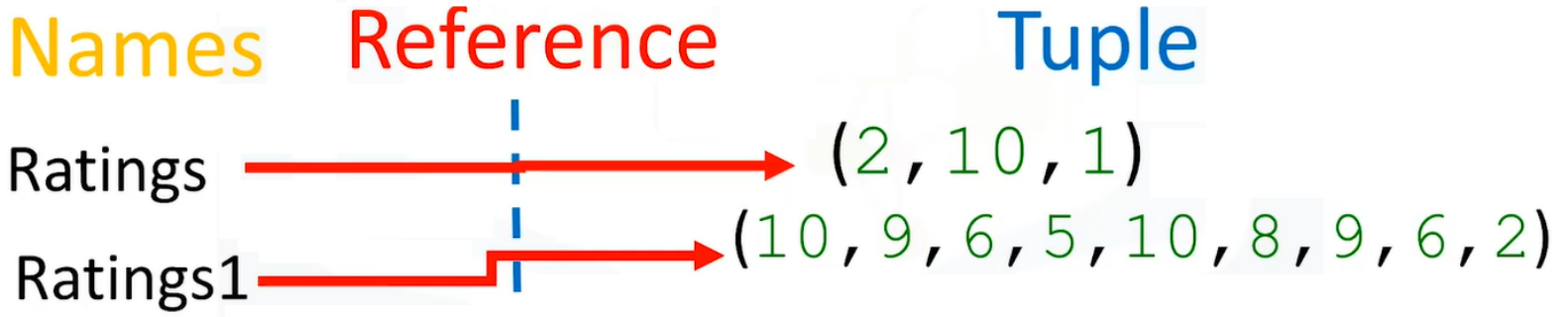
Tuple: Immutable

```
Ratings =(10,9,6,5,10,8,9,6,2)
Ratings1=Ratings
```

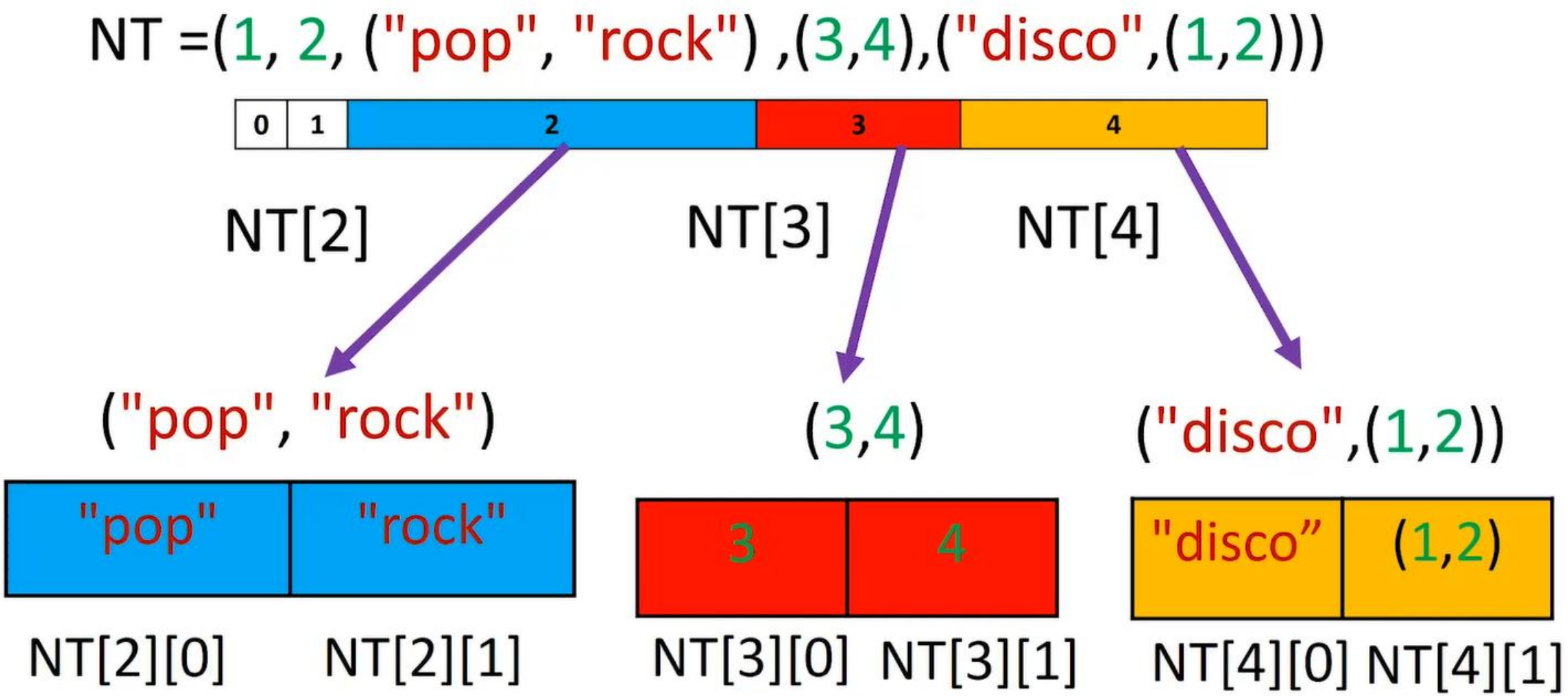
```
Rating    =4
```



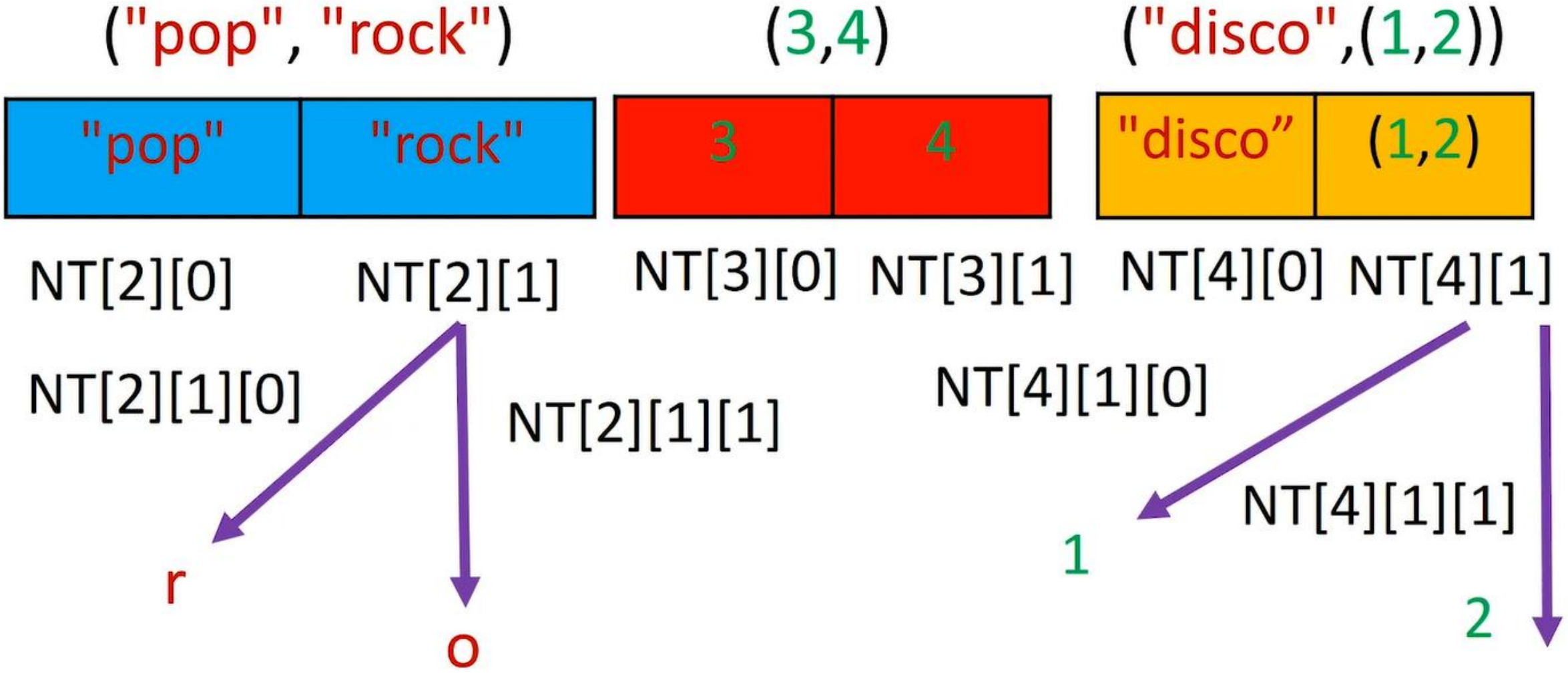

```
Ratings =(2,10,1)
```



Tuple: Nesting & Slicing



Tuple: Nesting & Slicing



Tuple: Packing & Unpacking

```
tuple_name = (value, value, ..., value)
```

- A way of packing multiple values into a variable

```
>>> x = 3
>>> y = -5
>>> p = (x, y, 42)
>>> p
(3, -5, 42)
```

```
name, name, ..., name = tuple_name
```

- Unpacking a tuple‘s contents in to multiple variables

```
>>> a, b, c = p
>>> a
3
>>> b
-5
>>> c
42
```

Trivia Time

Consider the following tuple:
say_what=('say',' what', 'you', 'will')

What is the result of the following **say_what[-1]** ?

- [x] 'will'
- [ ] 'what'
- [ ] 'you'
- [ ] 'say'

Trivia Time

Consider the following tuple **A=(1,2,3,4,5)**, what is the result of the following:**A[1:4]**:

- ◯ (2, 3, 4, 5)
- ◯ (3, 4, 5)
- ✗ (2, 3, 4)

Consider the following tuple **A=(1,2,3,4,5)**. What is the result of the following: **len(A)**

- ◯ 6
- ✗ 5
- ◯ 4

Practice Session: Jupyter Notebook

Lecture_2_Tuples_MorePractice

Task 2: 5 minutes

```
tuple1 = ("Orange", [10, 20, 30], (5, 15, 25))
```

a) Access the value of 20 from tuple1

```
tuple2 = (10, 20, 30, 40)
```

b) Unpack tuple2 into 4 variables a,b,c,d.

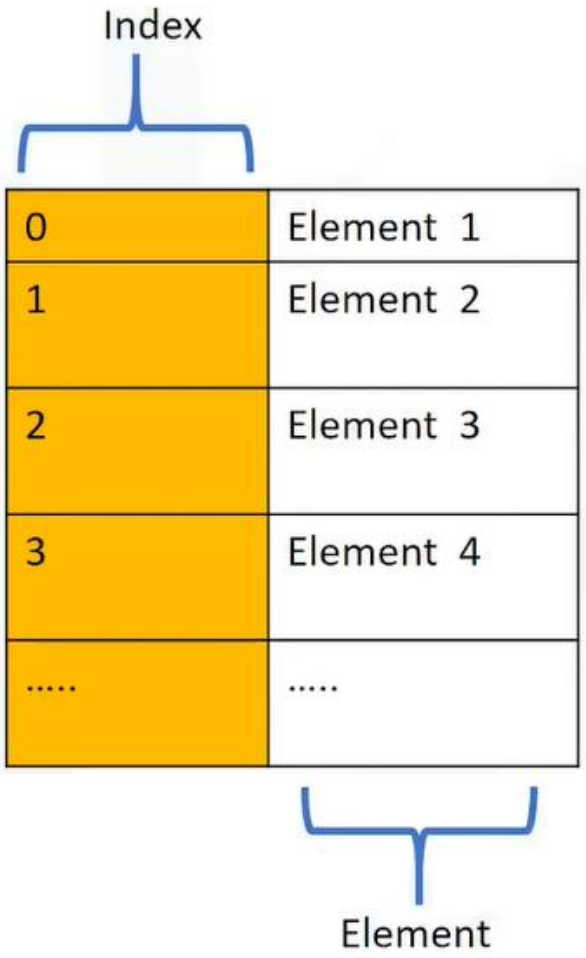
```
tuple3 = (11, 22)
tuple4 = (99, 88)
```

c) Swap the value of tuples

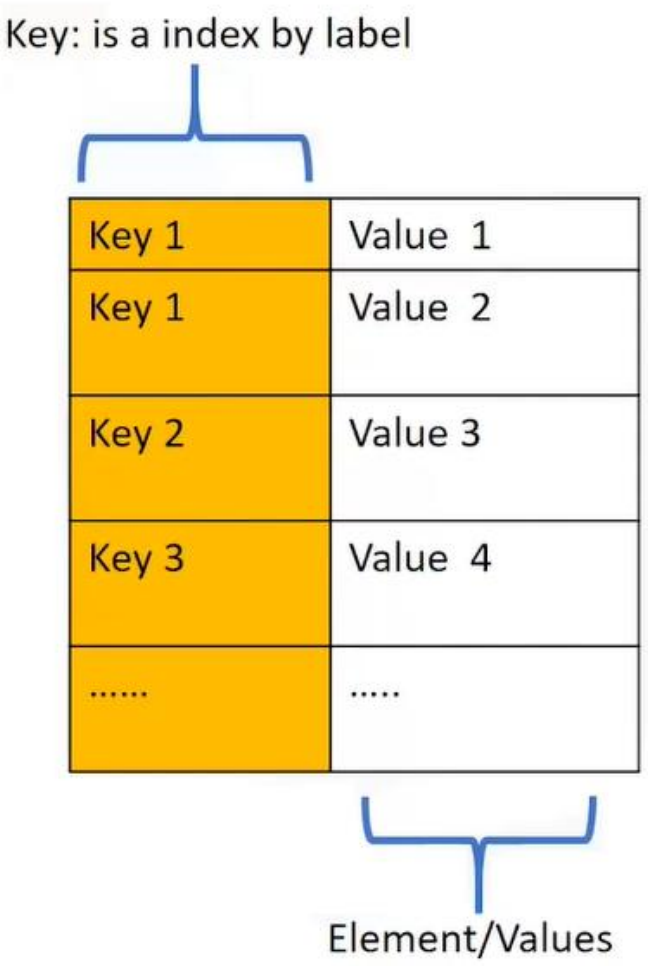
```
print(tuple1[1][1])



a, b, c, d = tuple2
print(a)
print(b)
print(c)
print(d)



tuple3, tuple4 = tuple4, tuple3
print(tuple4)
print(tuple3)
```

Dictionaries

List



Dictionary



Dictionaries

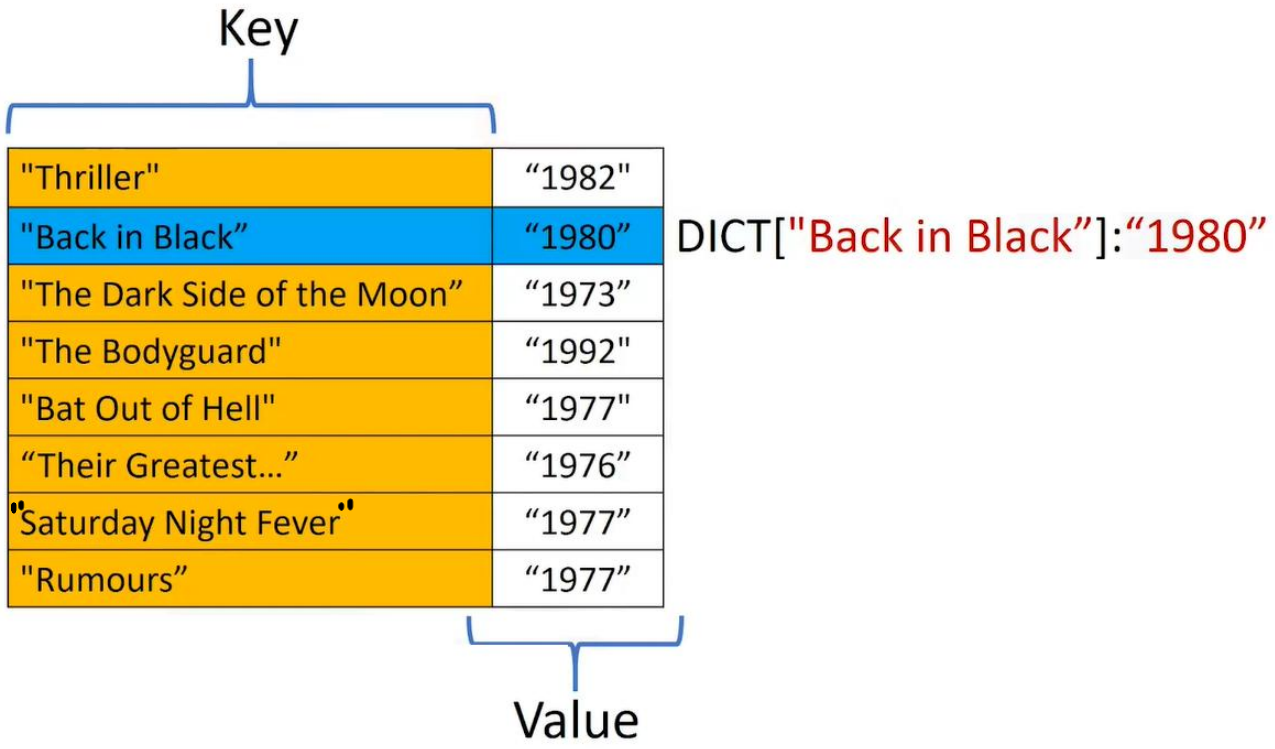
- Dictionaries are denoted with curly brackets {}.
- The keys have to be immutable and unique.
- The values can be immutable, mutable and duplicates.
- Each key and value pair is separated by a comma.
- For e.g:

{“key1”:1, “key2”:” 2”, “key3”: [3,2,3], “key4”: (2,5,6), “key5”: 3}

Dictionaries

DICT=

"Thriller"	“1982"
"Back in Black"	“1980”
"The Dark Side of the Moon”	“1973”
"The Bodyguard"	“1992"
"Bat Out of Hell"	“1977"
“Their Greatest…”	“1976”
"Saturday Night Fever"	“1977”
"Rumours”	“1977”



DICT["Back in Black”]:“1980”

Dictionaries

DICT=

"Thriller"	“1982"
"Back in Black"	“1980”
"The Dark Side of the Moon”	“1973”
"The Bodyguard"	“1992"
"Bat Out of Hell"	“1977"
“Their Greatest…”	“1976”
"Saturday Night Fever"	“1977”
"Rumours”	“1977”

"Thriller"	“1982"
"Back in Black”	“1980”
"The Dark Side of the Moon”	“1973”
"The Bodyguard"	“1992"
"Bat Out of Hell"	“1977"
“Their Greatest…”	“1976”
"Saturday Night Fever"	“1977”
"Rumors”	“1977”
"Graduation"	“2007”

DICT['Graduation']='2007'

Dictionaries

DICT=

"Thriller"	“1982"
"Back in Black”	“1980”
"The Dark Side of the Moon”	“1973”
"The Bodyguard"	“1992"
"Bat Out of Hell"	“1977"
“Their Greatest…”	“1976”
"Saturday Night Fever"	“1977”
"Rumors”	“1977”
"Graduation"	“2007”

"Back in Black”	“1980”
"The Dark Side of the Moon”	“1973”
"The Bodyguard"	“1992"
"Bat Out of Hell"	“1977"
“Their Greatest…”	“1976”
"Saturday Night Fever"	“1977”
"Rumors”	“1977”
"Graduation"	“2007”

del(DICT['Thriller'])

Dictionaries

DICT=

"Back in Black"	"1980"
"The Dark Side of the Moon"	"1973"
"The Bodyguard"	"1992"
"Bat Out of Hell"	"1977"
"Their Greatest..."	"1976"
"Saturday Night Fever"	"1977"
"Rumors"	"1977"

'The Bodyguard' in DICT

True

"Starboy" in DICT

False

DICT.keys()=["Back in Black", "The Dark Side of the Moon", "The Bodyguard", "Bat Out of Hell", "Their Greatest...","Saturday Night Fever", "Rumors"]

DICT.values() = ["1980", "1973", "1992", "1977", "1976", "1977"]

Trivia Time

What are the keys of the following dictionary: **{"a":1,"b":2}**

- ○ 1,2
- ✗ "a","b"

Consider the following Python Dictionary: **Dict={"A":1,"B":"2","C":[3,3,3],"D":(4,4,4),'E':5,'F':6}** What is the result of the following operation: **Dict["D"]**

- ○ [3,3,3]
- ✗ (4, 4, 4)
- ○ 1

Practice Session: Jupyter Notebook

Lecture_2_Dictionaries_MorePractice

Task 3: 7 minutes

```
keys = ['Ten', 'Twenty', 'Thirty’]
values = [10, 20, 30]
```

a) Make a dict from the two lists.

b) Extract the values of keys and values from res_dict.

c) Delete the key “Thirty” from res_dict.

d) Add new key “Fourty” having value 40 to res_dict.

```
res_dict = dict(zip(keys, values))
print(res_dict)
```

```
res_dict.keys()
res_dict.values()
```

```
del(re_dict[“Thirty”]
```

```
res_dict["Fourty"]=40
```

BREAK: 15Minz

Thank You for Attention